

DATABASE MANAGEMENT SYSTEM

Semester Answer Notes

MODULE - I - 5 marks

1. Explain Database Schema with Example

A database schema is the logical structure or blueprint of a database. It defines how data is organized, including tables, attributes, relationships, and constraints. The schema is created during database design and usually remains unchanged.

Example:

Student(StudentID, Name, Department, Age)

Here, Student is the table name and StudentID, Name, Department, and Age are attributes. This structure represents the schema of the database.

A sample record such as

(101, Rahul, CSE, 20)

is called an instance of the schema.

Thus, a database schema provides a framework for storing and organizing data efficiently.

2. Define (i) Database (ii) DBMS

Database:

A database is an organized collection of related data that can be easily accessed, managed, and updated. Examples include student databases, banking databases, and hospital databases.

DBMS:

A Database Management System (DBMS) is software used to create, store, retrieve, and manage databases.

Examples of DBMS are MySQL, Oracle, PostgreSQL, and SQL Server.

Major functions of a DBMS include data storage, security, backup and recovery, and data retrieval. Thus, a database stores data while a DBMS manages that data efficiently.

3. Discuss Five Main Advantages of DBMS over File System

DBMS provides several advantages over the traditional file system.

1. **Reduced Data Redundancy:** Duplicate data is minimized.
2. **Improved Consistency:** Data remains accurate and consistent.
3. **Better Security:** Access to data can be controlled using authorization.
4. **Concurrent Access:** Multiple users can access data simultaneously.

5. **Backup and Recovery:** Data can be restored after failures.

Therefore, DBMS provides efficient, secure, and reliable data management compared to file processing systems.

4. Explain the Role of Database Administrator

A Database Administrator (DBA) is the person responsible for managing and maintaining a database system. The DBA ensures that the database operates efficiently, securely, and reliably.

The main responsibilities of a DBA are:

- Database design and creation.
- User account and access control management.
- Backup and recovery of data.
- Performance monitoring and tuning.
- Maintaining data security and integrity.

Thus, the DBA plays a vital role in ensuring the smooth operation of a database system.

5. Explain Different Types of Database End Users

Database end users are people who interact with the database to perform various tasks.

The main types of end users are:

1. **Naive Users:** Use predefined applications without knowing database details.
2. **Casual Users:** Access the database occasionally using queries.
3. **Sophisticated Users:** Use advanced tools and write complex queries.
4. **Standalone Users:** Maintain personal databases using software packages.

Different users access databases according to their requirements and level of expertise.

6. Explain Database Instance with Example

A database instance is the collection of information stored in a database at a particular moment of time. It represents the actual data stored in the database.

Example:

Schema:

Student(ID, Name, Department)

Instance:

(101, Rahul, CSE)

(102, Priya, ECE)

The schema remains fixed, but the instance changes whenever records are inserted, deleted, or modified.

Thus, an instance represents the current state of a database.

7. Explain Logical Data Independence and Physical Data Independence

Data independence refers to the ability to modify one level of the database without affecting the next higher level.

Logical Data Independence

It is the ability to change the conceptual schema without affecting external views.

Example: Adding a new attribute such as Email to the Student table.

Physical Data Independence

It is the ability to change the physical storage structure without affecting the conceptual schema.

Example: Changing the indexing method.

Physical data independence is easier to achieve than logical data independence.

8. What is the Purpose of Database System? Describe the Three Levels of Data Abstraction

The main purpose of a database system is to store, manage, and retrieve data efficiently while reducing redundancy and maintaining consistency.

The three levels of data abstraction are:

1. **Internal Level:** Describes how data is physically stored.
2. **Conceptual Level:** Describes the logical structure of the entire database.
3. **External Level:** Describes how users view the data.

These levels provide data independence and simplify database management.

9. What are the Different Types of Schema?

A database system contains three types of schema.

1. **Physical Schema:** Describes how data is stored physically in memory.
2. **Logical Schema:** Describes the logical structure of tables and relationships.
3. **External Schema:** Describes different user views of the database.

These schemas help achieve data abstraction and independence in a database system.

10. Explain the Database Design Process

Database design is the process of creating a database structure that satisfies user requirements.

The main steps are:

1. Requirement collection and analysis.
2. Conceptual design using ER diagrams.

3. Logical database design.
4. Schema refinement and normalization.
5. Physical database design.
6. Database implementation and testing.

A proper design ensures efficient storage, retrieval, and maintenance of data.

11. Explain the Difference Between Metadata and Data Dictionary. What are the Different Data Models?

Metadata

Metadata is data about data. It describes the properties of data such as name, type, and size.

Example: StudentID is an integer attribute.

Data Dictionary

A data dictionary is a repository that stores metadata information about the database.

Different Data Models

1. Hierarchical Model
2. Network Model
3. Relational Model
4. Entity-Relationship Model
5. Object-Oriented Model

These models define how data is represented and organized in a database.

12. Discuss Database Users and Administrators

Database users are individuals who interact with the database to perform various tasks.

Types of database users include:

- Naive Users
- Casual Users
- Sophisticated Users
- Application Programmers

A Database Administrator (DBA) manages the database system and performs tasks such as security management, backup and recovery, performance tuning, and user authorization.

Both users and administrators contribute to the effective functioning of a database system.

13. Explain Database Languages with Examples

Database languages are used to define, manipulate, control, and manage data in a database.

DDL (Data Definition Language)

Used to define database structures.

```
CREATE TABLE Student(  
ID INT,  
Name VARCHAR(20)  
);
```

DML (Data Manipulation Language)

Used to insert, update, and delete data.

```
INSERT INTO Student VALUES(1,'Rahul');
```

DCL (Data Control Language)

Used for access control.

```
GRANT SELECT ON Student TO User1;
```

TCL (Transaction Control Language)

Used for transaction management.

```
COMMIT;  
ROLLBACK;
```

4. Concurrent Access

Multiple users can access the same database simultaneously. DBMS uses concurrency control techniques to prevent conflicts.

5. Backup and Recovery

DBMS provides automatic backup and recovery mechanisms to restore data after failures such as hardware crashes or accidental deletion.

Additional Advantages

- Data independence.
- Better data sharing.
- Improved integrity.
- Efficient query processing.

Hence, DBMS is more efficient, secure, and reliable than the traditional file processing system.

MODULE - I - 10 marks

1. Discuss Three Tier Database Architecture with Diagram

The Three Tier Database Architecture is a client-server architecture in which the application is divided into three independent layers. This architecture improves security, scalability, maintainability, and performance.

The three layers are:

1. Presentation Tier
2. Application Tier
3. Database Tier

1. Presentation Tier

The Presentation Tier is the user interface layer through which users interact with the system. It accepts user inputs and displays the results.

Examples: Web browsers, mobile applications, desktop applications.

2. Application Tier

The Application Tier contains business logic and processing rules. It receives requests from users, processes them, and communicates with the database server.

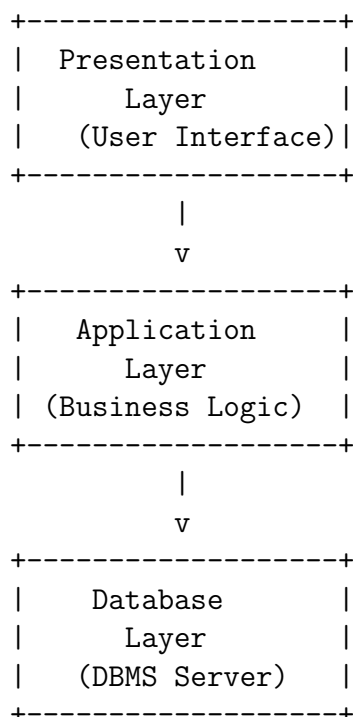
Examples: Java applications, Python applications, PHP applications.

3. Database Tier

The Database Tier stores and manages data. It executes SQL queries and returns results to the application layer.

Examples: MySQL, Oracle, PostgreSQL.

Diagram



Advantages

- Improved security.
- Easy maintenance.
- Better scalability.
- Efficient data management.
- Reduced system complexity.

Thus, the Three Tier Architecture separates user interface, business logic, and data storage, making database systems more efficient and secure.

2. Define Physical Schema, Logical Schema and Conceptual Schema with Suitable Diagram

A schema is the overall design of a database. The database architecture contains three types of schemas namely Physical Schema, Logical Schema, and Conceptual Schema.

1. Physical Schema

The Physical Schema describes how data is physically stored in the storage devices. It includes:

- File organization
- Storage structures
- Indexes
- Access paths

Example: Data stored in B+ Trees or Indexed Files.

2. Logical Schema

The Logical Schema describes the logical structure of the database. It specifies:

- Tables
- Attributes
- Relationships
- Constraints

Example:

Student(StudentID, Name, Department)

3. Conceptual Schema

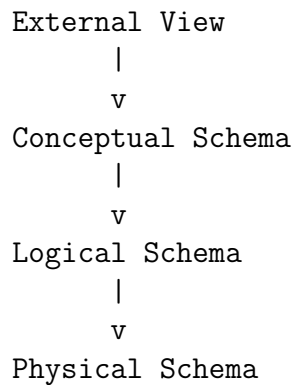
The Conceptual Schema provides the overall view of the entire database. It integrates all user views into a single global view.

It specifies:

- Entities
- Relationships

- Constraints

Diagram



Differences

- Physical Schema deals with storage details.
- Logical Schema deals with database structure.
- Conceptual Schema provides the complete view of the database.

Thus, all three schemas work together to provide data abstraction and independence.

3. Draw the Architecture of Database

Database Architecture describes how users interact with the database system through different levels of abstraction.

The architecture consists of three levels:

1. External Level
2. Conceptual Level
3. Internal Level

External Level

This level contains multiple user views. Different users can access different parts of the database.

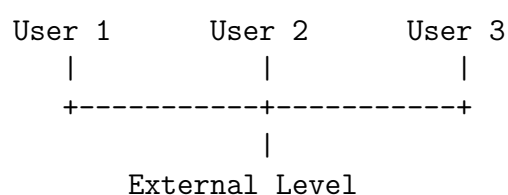
Conceptual Level

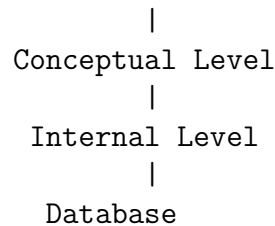
This level describes the complete logical structure of the database.

Internal Level

This level describes how data is physically stored in memory and storage devices.

Database Architecture Diagram





Advantages

- Data abstraction.
- Data independence.
- Improved security.
- Easy maintenance.

Hence, database architecture provides efficient data management through multiple abstraction levels.

4. What are the Two Types of Participation Constraint? What are Different Access Types in DML?

Participation Constraint

Participation constraint specifies whether every entity must participate in a relationship or not.

There are two types of participation constraints.

1. Total Participation

Every entity in the entity set must participate in at least one relationship.

Example:

Every employee must belong to a department.

Employee ===== Works_For ===== Department

(Double line indicates total participation.)

2. Partial Participation

Only some entities participate in the relationship.

Example:

Only some employees manage departments.

Employee ----- Manages ----- Department

(Single line indicates partial participation.)

Access Types in DML

Data Manipulation Language (DML) is used to access and manipulate data.

Main access types are:

1. Retrieval (SELECT)
2. Insertion (INSERT)

3. Modification (UPDATE)

4. Deletion (DELETE)

Examples:

```
SELECT * FROM Student;
```

```
INSERT INTO Student VALUES(...);
```

```
UPDATE Student  
SET Name='Rahul';
```

```
DELETE FROM Student  
WHERE ID=101;
```

Thus, participation constraints define relationship involvement while DML access types manipulate database data.

5. Differentiate File Processing System with Database Management System

A File Processing System stores data in separate files, whereas a DBMS stores data in a centralized database.

Feature	File Processing System	DBMS
Data Redundancy	High	Low
Data Consistency	Difficult to maintain	Easily maintained
Data Security	Limited	Strong security mechanisms
Data Sharing	Difficult	Easy
Backup and Recovery	Manual	Automatic support
Concurrency Control	Poor	Efficient
Data Independence	Not supported	Supported
Query Processing	Complex	Easy using SQL
Maintenance	Difficult	Easy
Cost	Low	Higher

Advantages of DBMS over File System

- Reduced redundancy.
- Improved consistency.
- Better security.
- Data independence.
- Backup and recovery support.

Therefore, DBMS is more efficient, secure, and reliable than the traditional file processing system.

MODULE - II - 5 Marks

1. Explain Derived Attribute and Multivalued Attribute with Example

Derived Attribute

A derived attribute is an attribute whose value can be calculated from another attribute.

Example:

Age can be derived from DateOfBirth.

$$Age = CurrentDate - DateOfBirth$$

In an ER diagram, a derived attribute is represented by a dashed ellipse.

Multivalued Attribute

A multivalued attribute can have more than one value for a single entity.

Example:

A student may have multiple phone numbers.

$$Student(Name, PhoneNo)$$

In an ER diagram, a multivalued attribute is represented by a double ellipse.

2. What is Key Attribute? Explain with Example

A key attribute is an attribute that uniquely identifies each entity in an entity set. No two entities can have the same value for a key attribute. It helps distinguish one record from another and is used for efficient data retrieval.

Example:

Consider the entity:

$$Student(StudentID, Name, Department)$$

Here, StudentID is a key attribute because each student has a unique ID. Even if two students have the same name, they cannot have the same StudentID.

Characteristics of a Key Attribute

- Uniquely identifies each entity.
- Cannot contain duplicate values.
- Helps maintain data integrity.
- Often used as the primary key of a table.

Thus, a key attribute plays an important role in uniquely identifying records and maintaining the correctness of a database.

3. What is Foreign Key? Explain with Example

A foreign key is an attribute or set of attributes in one table that refers to the primary key of another table. It is used to establish a relationship between two tables and maintain referential integrity.

Example:

Department Table:

Department(DeptID, DeptName)

Student Table:

Student(StudentID, Name, DeptID)

Here, DeptID in the Student table is a foreign key because it references the primary key DeptID of the Department table.

Advantages of Foreign Key

- Establishes relationships between tables.
- Maintains data consistency.
- Prevents invalid data entry.
- Supports referential integrity.

Therefore, foreign keys are essential for connecting related tables and ensuring database consistency.

4. Explain Hierarchical Data Model with Example

The Hierarchical Data Model is one of the oldest database models. In this model, data is organized in the form of a tree-like structure consisting of parent-child relationships. Each parent record can have multiple child records, but each child record can have only one parent.

Example:

```
Company
|
+-- Department
    |
    +-- Employee
```

In the above example, a company contains departments, and each department contains employees.

Advantages

- Simple and easy to understand.
- Fast data retrieval.
- Suitable for one-to-many relationships.

Disadvantages

- Difficult to represent many-to-many relationships.
- Complex restructuring of data.

Thus, the hierarchical model organizes data efficiently using a parent-child structure.

5. Define Entity, Attribute, Entity Set and Relationship with Example

Entity

An entity is a real-world object that can be uniquely identified.

Example: Student, Employee, Course.

Attribute

An attribute is a property or characteristic of an entity.

Example: StudentID, Name, Age.

Entity Set

An entity set is a collection of similar entities.

Example: All students in a college form a Student entity set.

Relationship

A relationship represents an association between two or more entities.

Example:

Student ----- Enrolled_In ----- Course

Here, the relationship "Enrolled_In" connects Student and Course entities.

In an ER diagram, a multivalued attribute is represented using a double ellipse.

These concepts form the basic building blocks of the Entity Relationship (ER) model.

6. Draw ER Diagram for Ternary Relationship Set with Suitable Example

A ternary relationship is a relationship that involves three entity sets simultaneously. It is used when the relationship among three entities cannot be represented using separate binary relationships.

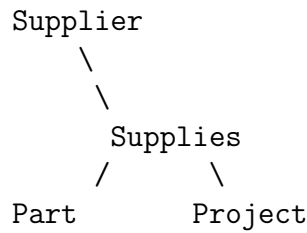
Example

A supplier supplies a particular part to a project.

The participating entities are:

- Supplier
- Part
- Project

ER representation:



In this relationship, information about Supplier, Part, and Project is stored together.

Ternary relationships are useful for modeling complex real-world situations where three entities interact at the same time.

7. What Does the Cardinality Ratio Specify? List Any Eight Applications of DBMS

Cardinality ratio specifies the number of entities to which another entity can be associated through a relationship. It defines the mapping between entity sets.

The four types of cardinality ratios are:

1. One-to-One (1:1)
2. One-to-Many (1:M)
3. Many-to-One (M:1)
4. Many-to-Many (M:N)

Applications of DBMS

1. Banking Systems
2. Railway Reservation Systems
3. Library Management Systems
4. Hospital Management Systems
5. Airline Reservation Systems
6. E-Commerce Applications
7. Educational Institutions
8. Telecommunication Systems

Cardinality ratios help in designing relationships correctly, while DBMS applications are found in almost every modern organization.

8. Define Relational Database. Explain Data Representation in RDBMS

A Relational Database is a database that stores data in the form of tables called relations. Each table consists of rows and columns, and relationships among tables are established using keys.

Example

StudentID	Name	Department
101	Rahul	CSE
102	Priya	ECE

In an RDBMS:

- Table is called a Relation.
- Row is called a Tuple.
- Column is called an Attribute.
- Degree refers to the number of attributes.
- Cardinality refers to the number of tuples.

Examples of RDBMS include MySQL, Oracle, PostgreSQL, and SQL Server.

Thus, an RDBMS represents data in a structured tabular form, making storage and retrieval easy.

9. What is Referential Integrity Constraint? How is it Related to Foreign Key? Explain with Example

Referential Integrity is a constraint that ensures consistency between related tables. It states that a foreign key value in one table must either match an existing primary key value in another table or be NULL.

Example

Department Table:

Department(DeptID, DeptName)

Student Table:

Student(StudentID, Name, DeptID)

Here, DeptID in the Student table is a foreign key referencing DeptID in the Department table.

If DeptID = 10 does not exist in the Department table, then a student record with DeptID = 10 cannot be inserted.

Benefits

- Maintains data consistency.
- Prevents invalid references.

- Preserves relationships between tables.
- Improves data integrity.

Therefore, referential integrity is enforced using foreign keys and ensures that relationships between tables remain valid.

MODULE - II - 10 Marks

1. Define Weak Entity Set, Candidate Key, Primary Key and Foreign Key

Weak Entity Set

A weak entity set is an entity set that cannot be uniquely identified by its own attributes. It depends on another entity set called the owner entity set for identification.

Example: Dependent of an Employee.

A weak entity is represented by a double rectangle in an ER diagram.

Candidate Key

A candidate key is an attribute or a set of attributes that can uniquely identify each tuple in a relation.

Example:

Student(StudentID, Email, Name)

Both StudentID and Email can uniquely identify a student and are candidate keys.

Primary Key

A primary key is the candidate key selected to uniquely identify tuples in a table.

Example:

Student(StudentID, Name, Department)

StudentID is the primary key.

Properties:

- Unique
- Cannot be NULL
- Identifies each record uniquely

Foreign Key

A foreign key is an attribute in one relation that refers to the primary key of another relation.

Example:

Department(DeptID, DeptName)

Student(StudentID, Name, DeptID)

DeptID in Student is a foreign key referencing Department.

Thus, keys play an important role in uniquely identifying records and maintaining relationships among tables.

3. What is the Difference Between Hierarchical and Network Model? Explain with Example

Hierarchical and Network models are early database models used to organize data.

Hierarchical Model

Data is organized in a tree structure. Each child has only one parent.

Example:

```
Company
|
Department
|
Employee
```

Network Model

Data is organized as a graph. A child can have multiple parents.

Example:

```
Student
| \
| \
| \
Course1 Course2
```

Differences

Feature	Hierarchical	Network
Structure	Tree	Graph
Parent Relationship	One Parent	Multiple Parents
Complexity	Simple	Complex
Flexibility	Less Flexible	More Flexible

The network model is more flexible because it supports many-to-many relationships.

4. Explain About Integrity Constraints Over Relations

Integrity constraints are rules that ensure the accuracy and consistency of data stored in a database.

The major integrity constraints are:

1. Domain Constraint

Specifies the permissible values for an attribute.

Example:

Age must be between 0 and 120.

2. Key Constraint

Ensures that key attributes contain unique values.

Example:

StudentID must be unique.

3. Entity Integrity Constraint

States that the primary key cannot contain NULL values.

Example:

StudentID cannot be NULL.

4. Referential Integrity Constraint

Ensures that a foreign key value matches a primary key value in the referenced relation.

Example:

DeptID in Student must exist in Department.

Advantages

- Maintains data consistency.
- Prevents invalid data entry.
- Improves database reliability.

Thus, integrity constraints help maintain correctness and consistency of database information.

5. Draw the ER Diagram for a Company Database

The company needs to store information about Employees, Departments, and Children of Employees.

Entities

1. Employee(SSN, Salary, Phone)
2. Department(DNo, DName, Budget)
3. Child(Name, Age)

Relationships

1. Employees work in Departments.
2. Each Department is managed by an Employee.
3. Employees can have Children.

Key Features

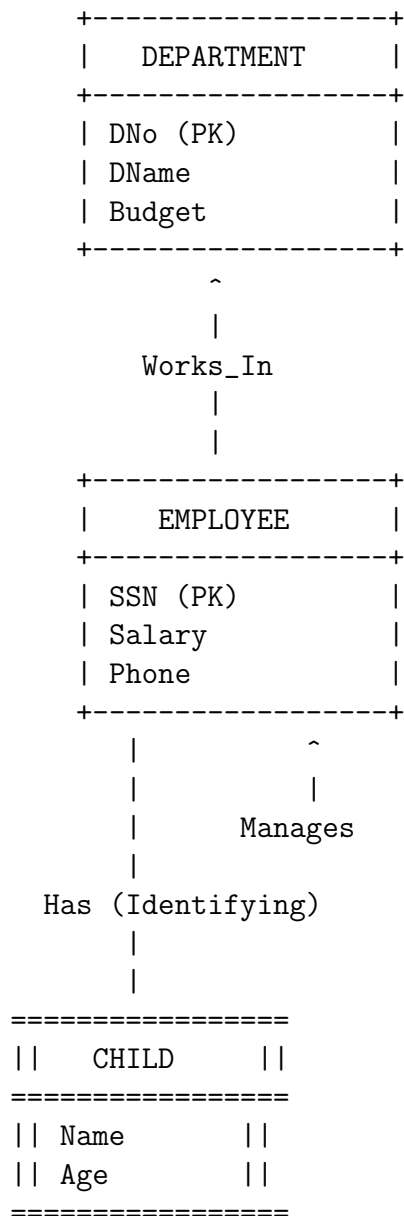
- SSN is the primary key of Employee.
- DNo is the primary key of Department.
- Child is a weak entity.
- Child is identified by the combination of Employee SSN and Child Name.
- The relationship between Employee and Child is an identifying relationship.
- If an Employee leaves the company, information about the Child is also removed.

Cardinality

- Department : Employee = 1 : N

- Employee : Child = 1 : N
- Employee : Department (Manager) = 1 : 1

ER Diagram



(Weak Entity)

Conclusion

Employee and Department are strong entities, whereas Child is a weak entity that depends on Employee for identification. The ER model stores information about employees, departments, department managers, and employee children efficiently.

6. Create ER Diagram for the Following

Given Conditions:

1. Each company operates four departments.
2. Each department belongs to exactly one company.
3. Each department employs one or more employees.
4. Each employee works for exactly one department.
5. Each employee may or may not have one or more dependents.

Entities

1. Company
2. Department
3. Employee
4. Dependent

Relationships and Cardinalities

1. Company – Department

A company operates four departments and each department belongs to one company.

Cardinality:

$$Company : Department = 1 : 4$$

2. Department – Employee

Each department employs one or more employees and each employee works for only one department.

Cardinality:

$$Department : Employee = 1 : N$$

3. Employee – Dependent

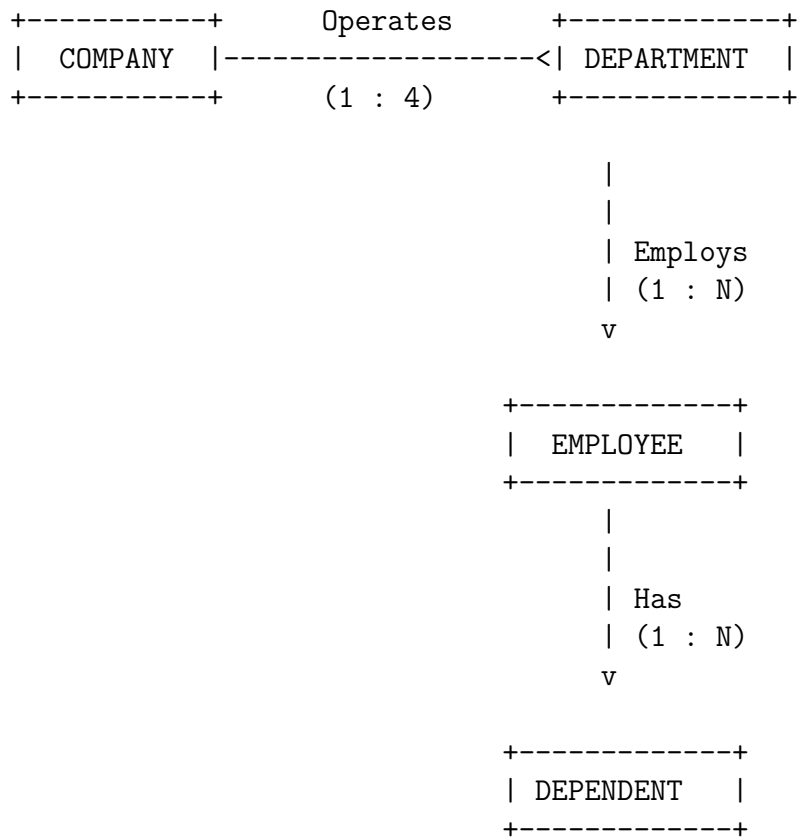
An employee may have zero, one, or many dependents.

Cardinality:

$$Employee : Dependent = 1 : N$$

Participation of Dependent is total because a dependent cannot exist without an employee.

ER Diagram



Explanation

A Company entity is related to the Department entity through the relationship “Operates”. Each company operates exactly four departments.

A Department entity is related to the Employee entity through the relationship “Employs”. One department can employ many employees, but an employee can work in only one department.

An Employee entity is related to the Dependent entity through the relationship “Has”. An employee may have multiple dependents or no dependent at all. A dependent cannot exist independently and is associated with exactly one employee.

Hence, the ER diagram consists of four entities: Company, Department, Employee, and Dependent with cardinalities 1:4, 1:N, and 1:N respectively.

MODULE - III - 5 Marks

1. Explain Data Definition Language (DDL) with Suitable Commands

Data Definition Language (DDL) is a category of SQL commands used to define and manage the structure of database objects such as tables, views, and indexes. DDL commands affect the database schema rather than the actual data.

The commonly used DDL commands are:

- CREATE
- ALTER
- DROP
- TRUNCATE
- RENAME

Examples

Creating a table:

```
CREATE TABLE Student(
ID INT,
Name VARCHAR(20)
);
```

Modifying a table:

```
ALTER TABLE Student
ADD Age INT;
```

Deleting a table:

```
DROP TABLE Student;
```

Thus, DDL commands are used to create, modify, and delete database structures.

2. Write Suitable SQL Query to Display Minimum, Maximum, Average and Total Salaries Earned by Employees

Aggregate functions are used to perform calculations on a group of values and return a single result. They are commonly used in SQL for data analysis and reporting.

Assume there is an Employee table containing a Salary attribute.

SQL Query

```
SELECT
MIN(Salary) AS MinimumSalary,
MAX(Salary) AS MaximumSalary,
AVG(Salary) AS AverageSalary,
SUM(Salary) AS TotalSalary
FROM Employee;
```

Explanation

- MIN() returns the smallest salary.
- MAX() returns the largest salary.
- AVG() returns the average salary.
- SUM() returns the total salary of all employees.

These functions help summarize salary information efficiently.

3. Explain Data Manipulation Language (DML) with Suitable Commands

Data Manipulation Language (DML) consists of SQL commands used to retrieve, insert, update, and delete data stored in database tables.

Unlike DDL, DML works on the data itself rather than the database structure.

The main DML commands are:

- SELECT
- INSERT
- UPDATE
- DELETE

Examples

Insert data:

```
INSERT INTO Student  
VALUES(101, 'Rahul');
```

Update data:

```
UPDATE Student  
SET Name='Anik'  
WHERE ID=101;
```

Delete data:

```
DELETE FROM Student  
WHERE ID=101;
```

Thus, DML commands are used for managing and manipulating records stored in database tables.

4. Explain Projection Operation in Relational Algebra with Suitable Example

Projection is a unary relational algebra operation used to select specific attributes (columns) from a relation. It is represented by the symbol π .

The projection operation removes unwanted columns and displays only the required attributes.

Suppose the relation is:

Student(ID, Name, Department)

To display only Name and Department:

$\pi_{Name, Department}(Student)$

Result:

Name	Department
Rahul	CSE
Priya	ECE

Projection helps reduce unnecessary information and makes query results easier to understand.

5. State the Difference Between Cartesian Product and Natural Join in Relational Algebra

Cartesian Product combines every tuple of one relation with every tuple of another relation. It is represented by the symbol $\times$.

Natural Join combines tuples from two relations based on matching values of common attributes. It is represented by the symbol $\bowtie$.

Differences

Feature	Cartesian Product	Natural Join
Symbol	$\times$	$\bowtie$
Condition	No condition	Uses common attributes
Result Size	Large	Smaller
Meaningful Output	Not always	Yes

Thus, Natural Join is generally more useful because it returns only matching tuples.

6. Give an Example of How to Use the Rename Operation in Relational Algebra

The Rename operation is used to rename a relation or its attributes. It is represented by the symbol ρ .

It is useful when dealing with complex queries and self-joins.

Suppose there is a relation:

$$Student(ID, Name, Department)$$

To rename the relation Student as S:

$$\rho_S(Student)$$

To rename attributes:

$$\rho_{S(SID, SName, Dept)}(Student)$$

Advantages

- Improves readability of queries.
- Avoids ambiguity.
- Useful in self-join operations.
- Provides temporary names for relations.

Therefore, the rename operator helps simplify relational algebra expressions and query representation.

7. Explain the Following: (i) Data Definition Language (ii) Data Control Language (iii) Views

(i) Data Definition Language (DDL)

Data Definition Language (DDL) consists of SQL commands used to define and modify the structure of database objects such as tables, views, and indexes.

Common DDL commands are:

- CREATE
- ALTER
- DROP
- TRUNCATE

Example:

```
CREATE TABLE Student(  
ID INT,  
Name VARCHAR(20)  
);
```

(ii) Data Control Language (DCL)

Data Control Language (DCL) is used to control access permissions in a database.

Common DCL commands are:

- GRANT
- REVOKE

Example:

```
GRANT SELECT  
ON Student  
TO User1;
```

(iii) Views

A view is a virtual table created from one or more tables. It does not store data physically but displays data from underlying tables.

Example:

```
CREATE VIEW CS_Students AS  
SELECT *  
FROM Student  
WHERE Department='CSE';
```

Views improve security and simplify complex queries.

8. What is a View? How Can It Be Created? Explain with an Example

A view is a virtual table that contains data derived from one or more base tables. The data is not stored separately; it is generated whenever the view is accessed.

Advantages of Views

- Improves security.
- Simplifies complex queries.
- Provides customized data access.
- Hides unnecessary information.

Syntax

```
CREATE VIEW view_name AS
SELECT column1,column2
FROM table_name
WHERE condition;
```

Example

```
CREATE VIEW CSE_Students AS
SELECT ID, Name
FROM Student
WHERE Department='CSE';
```

Whenever the view is queried, only CSE student information is displayed.
Thus, views provide a convenient and secure way of accessing database information.

9. What is a Join Operator? Explain Conditional Join and Natural Join with Syntax and Example

A join operator combines tuples from two or more relations based on a specified condition. It helps retrieve related information stored in different tables.

Conditional Join (Theta Join)

A conditional join combines tuples satisfying a specific condition.

Syntax:

$$R \bowtie_{condition} S$$

Example:

$$Student \bowtie_{Student.DeptID=Department.DeptID} Department$$

Natural Join

A natural join automatically joins two relations using common attributes with the same name.

Syntax:

$$R \bowtie S$$

Example:

$$Student \bowtie Department$$

Differences

- Conditional join requires an explicit condition.
- Natural join automatically uses common attributes.
- Natural join eliminates duplicate columns.

Joins are widely used to combine data from multiple tables.

10. How Can We Compare Using NULL Values? Explain About Logical Connectives with Examples

NULL represents missing, unknown, or unavailable data in a database. It is different from zero and empty strings.

NULL cannot be compared using ordinary comparison operators such as = or <.

To test NULL values, special operators are used.

Examples

Check for NULL:

```
SELECT *
FROM Employee
WHERE Salary IS NULL;
```

Check for NOT NULL:

```
SELECT *
FROM Employee
WHERE Salary IS NOT NULL;
```

Logical Connectives

Logical connectives are used to combine multiple conditions in SQL.

- AND
- OR
- NOT

Examples:

```
SELECT *
FROM Student
WHERE Department='CSE'
AND Age>18;
```

```
SELECT *
FROM Student
WHERE Department='CSE'
OR Department='ECE';
```

```
SELECT *
FROM Student
WHERE NOT Department='CSE';
```

Logical connectives help form complex conditions in SQL queries.

MODULE - III - 10 Marks

1. Discuss Different Types of Outer Join with Example

A join operation is used to combine rows from two or more relations based on a common attribute. In many situations, some tuples in one relation may not have matching tuples in another relation. To preserve such tuples, Outer Join operations are used.

An Outer Join returns matching tuples as well as non-matching tuples by filling missing values with NULL.

The three types of Outer Joins are:

1. Left Outer Join

The Left Outer Join returns all tuples from the left relation and only the matching tuples from the right relation. If there is no matching tuple in the right relation, NULL values are inserted.

Example:

Employee

EmpID	Name
1	Rahul
2	Priya
3	Amit

Department

EmpID	Dept
1	CSE
2	ECE

Result of Left Outer Join:

EmpID	Name	Dept
1	Rahul	CSE
2	Priya	ECE
3	Amit	NULL

2. Right Outer Join

The Right Outer Join returns all tuples from the right relation and matching tuples from the left relation.

If a matching tuple is not found in the left relation, NULL values are inserted.

Example:

EmpID	Name	Dept
1	Rahul	CSE
2	Priya	ECE
4	NULL	ME

3. Full Outer Join

The Full Outer Join returns all tuples from both relations. Matching tuples are combined and unmatched tuples from either relation are also included.

Result:

EmpID	Name	Dept
1	Rahul	CSE
2	Priya	ECE
3	Amit	NULL
4	NULL	ME

Advantages of Outer Join

- Preserves unmatched records.
- Useful for reporting and analysis.
- Helps identify missing relationships.
- Provides complete information from participating relations.

Conclusion

Outer Join is an important relational operation used when all records from one or both relations must be displayed regardless of whether matching tuples exist. The three types of Outer Joins are Left Outer Join, Right Outer Join, and Full Outer Join.

2. Discuss Different Types of Integrity Constraints with Example

Integrity constraints are rules applied to a database to ensure the accuracy, consistency, and reliability of data. They prevent invalid data from being entered into the database and help maintain data integrity.

The main types of integrity constraints are:

1. Domain Constraint

A domain constraint specifies the permissible values that an attribute can hold. The value entered must belong to the defined domain.

Example:

```
Age INT CHECK(Age > 0)
```

Here, Age cannot be negative.

2. Key Constraint

A key constraint ensures that certain attributes uniquely identify each tuple in a relation.

Example:

Student(StudentID, Name, Department)

StudentID must be unique for every student.

3. Entity Integrity Constraint

This constraint states that the primary key of a relation cannot be NULL because every tuple must be uniquely identifiable.

Example:

In the Student table, StudentID cannot contain NULL values.

4. Referential Integrity Constraint

Referential integrity ensures that a foreign key value in one relation matches a primary key value in another relation.

Example:

Department(DeptID, DeptName)

Student(StudentID, Name, DeptID)

Here, DeptID in Student must exist in the Department table.

5. Check Constraint

A check constraint restricts values based on a specified condition.

Example:

CHECK (Salary > 1000)

This ensures that salary values greater than 1000 are accepted.

Advantages of Integrity Constraints

- Maintain data consistency.
- Prevent invalid data entry.
- Improve database reliability.
- Preserve relationships among tables.

Conclusion

Integrity constraints are essential for maintaining the correctness and consistency of data in a database. The major constraints are Domain, Key, Entity Integrity, Referential Integrity, and Check constraints.

3. Relational Algebra Operations

(i) Display Regno and Section from the Students table where Branch is EE

To solve this query, we first select the tuples whose Branch is EE and then project the required attributes Regno and Section.

$$\pi_{Regno,Section}(\sigma_{Branch='EE'}(Students))$$

Explanation

- σ (Selection) chooses tuples where Branch = EE.
- π (Projection) displays only Regno and Section columns.

(ii) Select tuples from the Book table where Topic is “Database” and Author is “Ram”

This query requires selecting tuples that satisfy both conditions.

$$\sigma_{Topic='Database' \wedge Author='Ram'}(Book)$$

Explanation

- σ (Selection) retrieves tuples satisfying the given condition.
- The logical operator $\wedge$ (AND) ensures that both conditions are true.

Conclusion

Selection and Projection are basic relational algebra operations. Selection filters rows based on conditions, while Projection displays only the required columns.

4. What are Aggregate Functions? Explain Five Built-in Aggregate Functions

Aggregate functions are special SQL functions that perform calculations on a set of values and return a single result. They are commonly used in queries involving numerical analysis, reports, and summaries.

Aggregate functions operate on a group of rows rather than on individual rows.

The five commonly used aggregate functions are:

1. COUNT()

The COUNT() function returns the number of rows in a table.

Example:

```
SELECT COUNT(*) FROM Student;
```

This query returns the total number of students.

2. SUM()

The SUM() function returns the total sum of values in a numeric column.

Example:

```
SELECT SUM(Salary)
FROM Employee;
```

This query calculates the total salary of all employees.

3. AVG()

The AVG() function returns the average value of a numeric column.

Example:

```
SELECT AVG(Salary)
FROM Employee;
```

This query calculates the average salary of employees.

4. MAX()

The MAX() function returns the highest value in a column.

Example:

```
SELECT MAX(Salary)
FROM Employee;
```

This query returns the highest salary among employees.

5. MIN()

The MIN() function returns the lowest value in a column.

Example:

```
SELECT MIN(Salary)
FROM Employee;
```

This query returns the minimum salary among employees.

Advantages of Aggregate Functions

- Simplify data analysis.
- Generate summary reports.
- Reduce query complexity.
- Improve decision making.

Conclusion

Aggregate functions are important SQL tools used to summarize and analyze data. The most commonly used aggregate functions are COUNT(), SUM(), AVG(), MAX(), and MIN().

5. Explain the Structure of Basic Form of an SQL Query. What is Cursor in Database? How Can You Create a User Defined Cursor to Store All Manager Details in SQL?

Basic Structure of an SQL Query

SQL (Structured Query Language) is used to retrieve and manipulate data stored in a database. The basic structure of an SQL query consists of three clauses: SELECT, FROM, and WHERE.

General Syntax:

```
SELECT column_name  
FROM table_name  
WHERE condition;
```

Example:

```
SELECT Name, Department  
FROM Student  
WHERE Department='CSE';
```

Explanation

- SELECT specifies the attributes to be displayed.
- FROM specifies the relation from which data is retrieved.
- WHERE specifies the condition for selecting tuples.

Additional clauses such as GROUP BY, HAVING, and ORDER BY can also be used in SQL queries.

Cursor in Database

A cursor is a database object used to process query results one row at a time. It is useful when row-by-row processing is required instead of handling the entire result set at once.

The main steps involved in cursor processing are:

1. Declare the cursor.
2. Open the cursor.
3. Fetch rows from the cursor.
4. Close the cursor.
5. Deallocate the cursor.

User Defined Cursor to Retrieve Manager Details

```
DECLARE ManagerCursor CURSOR
FOR
SELECT *
FROM Employee
WHERE Designation='Manager';
```

Open the cursor:

```
OPEN ManagerCursor;
```

Fetch records:

```
FETCH NEXT
FROM ManagerCursor;
```

Close the cursor:

```
CLOSE ManagerCursor;
```

Release memory:

```
DEALLOCATE ManagerCursor;
```

Advantages of Cursor

- Processes records one at a time.
- Useful for complex row-by-row operations.
- Provides greater control over query results.

Conclusion

SQL queries are used to retrieve and manipulate data efficiently. A cursor is a special database object that allows row-by-row processing of query results and is particularly useful for handling manager details or other specific records individually.

6. Consider the Given Relations

Relations

STUDENT(*SID*, *Name*, *Dept*, *Age*)

COURSE(*CID*, *CName*, *Dept*)

ENROLL(*SID*, *CID*, *Marks*)

Write Relational Algebra expressions for the following queries.

(i) Find Names of All Students

To display the names of all students, the projection operator is used.

$$\pi_{Name}(STUDENT)$$

Explanation

The projection operator selects only the Name attribute from the STUDENT relation.

(ii) Find Names of Students Whose Age is Greater Than 20

First select students whose age is greater than 20 and then display their names.

$$\pi_{Name}(\sigma_{Age>20}(STUDENT))$$

Explanation

- Selection chooses tuples where Age $\geq$ 20.
- Projection displays only the Name attribute.

(iii) Find Students Enrolled in DBMS

First find the course DBMS, then join COURSE, ENROLL, and STUDENT relations.

$$\pi_{Name}(STUDENT \bowtie ENROLL \bowtie \sigma_{CName='DBMS'}(COURSE))$$

Explanation

The join operation connects students with enrolled courses and selects only those enrolled in DBMS.

(iv) Find Students Who Are Not Enrolled in Any Course

First find all student IDs and subtract the IDs of enrolled students.

$$\pi_{SID}(STUDENT) - \pi_{SID}(ENROLL)$$

To display names:

$$\pi_{Name}(STUDENT \bowtie (\pi_{SID}(STUDENT) - \pi_{SID}(ENROLL)))$$

Explanation

The set difference operator identifies students who do not appear in the ENROLL relation.

(v) Find Students Who Are Enrolled in All Courses

This query requires the Division operator.

$$\pi_{SID,CID}(ENROLL) \div \pi_{CID}(COURSE)$$

To display names:

$$\pi_{Name}(STUDENT \bowtie (\pi_{SID,CID}(ENROLL) \div \pi_{CID}(COURSE)))$$

Explanation

The division operator returns those students who are enrolled in every course present in the COURSE relation.

Conclusion

The above queries use various relational algebra operations such as Selection (σ), Projection (π), Join ($\bowtie$), Difference ($-$), and Division ($\div$) to retrieve the required information from the given relations.

MODULE - IV - 5 Marks

1. Explain Functional Dependency with Suitable Example

A Functional Dependency (FD) is a relationship between two attributes in a relation. It specifies that the value of one attribute uniquely determines the value of another attribute.

If attribute B depends on attribute A, then it is represented as:

$$A \rightarrow B$$

This means that for each value of A, there is exactly one corresponding value of B.

Example

Consider the relation:

$$Student(StudentID, Name, Department)$$

Since each StudentID uniquely identifies a student,

$$StudentID \rightarrow Name, Department$$

Here, StudentID determines both Name and Department.

Functional dependencies are important in normalization because they help reduce redundancy and maintain data consistency.

2. Explain Transitive Dependency with Suitable Example

A transitive dependency occurs when one non-key attribute depends on another non-key attribute through a key attribute.

If

$$A \rightarrow B$$

and

$$B \rightarrow C$$

then

$$A \rightarrow C$$

is called a transitive dependency.

Example

Consider:

$$Student(StudentID, DeptID, DeptName)$$

Functional dependencies:

$$StudentID \rightarrow DeptID$$

$$DeptID \rightarrow DeptName$$

Therefore,

$$StudentID \rightarrow DeptName$$

This is a transitive dependency.

Transitive dependencies create redundancy and are removed during normalization to achieve Third Normal Form (3NF).

3. Define Third Normal Form (3NF) with Example

A relation is said to be in Third Normal Form (3NF) if it is in Second Normal Form (2NF) and contains no transitive dependency.

In 3NF, non-key attributes must depend only on the primary key.

Example

Consider:

$$Student(StudentID, DeptID, DeptName)$$

Functional dependencies:

$$StudentID \rightarrow DeptID$$

$$DeptID \rightarrow DeptName$$

Since DeptName depends on DeptID rather than directly on StudentID, the relation is not in 3NF.

To convert it into 3NF:

Student(StudentID, DeptID)

Department(DeptID, DeptName)

Thus, 3NF removes transitive dependencies and reduces redundancy.

4. What is Redundancy? Explain with Suitable Example. What are the Anomalies Created by Data Redundancy?

Redundancy refers to the unnecessary repetition of data in a database. It increases storage requirements and may lead to inconsistencies.

Example

SID	Name	DeptName
101	Rahul	CSE
102	Priya	CSE
103	Amit	CSE

Here, the department name CSE is repeated multiple times.

Anomalies Caused by Redundancy

1. **Update Anomaly** – Updating the same information in multiple places.
2. **Insertion Anomaly** – Difficulty inserting data without additional information.
3. **Deletion Anomaly** – Loss of useful information when deleting a record.

Normalization is used to reduce redundancy and eliminate these anomalies.

“latex

2. Explain Transitive Dependency with Suitable Example

A transitive dependency occurs when a non-key attribute depends on another non-key attribute through a primary key. In other words, if attribute A determines B and B determines C, then A indirectly determines C.

Mathematically,

$$A \rightarrow B$$

and

$$B \rightarrow C$$

implies

$$A \rightarrow C$$

This is called transitive dependency.

Example

Consider the relation:

$$Student(StudentID, DeptID, DeptName)$$

Functional Dependencies:

$$StudentID \rightarrow DeptID$$

$$DeptID \rightarrow DeptName$$

Therefore,

$$StudentID \rightarrow DeptName$$

Since DeptName depends on StudentID through DeptID, a transitive dependency exists.

Transitive dependencies lead to redundancy and update anomalies. They are removed during normalization to achieve Third Normal Form (3NF).

3. Define Third Normal Form (3NF) with Example

A relation is said to be in Third Normal Form (3NF) if it is already in Second Normal Form (2NF) and contains no transitive dependency. In 3NF, every non-key attribute must depend directly on the primary key.

Conditions for 3NF

- The relation must be in 2NF.
- No non-key attribute should depend on another non-key attribute.

Example

Consider the relation:

$$Student(StudentID, DeptID, DeptName)$$

Functional Dependencies:

$$StudentID \rightarrow DeptID$$

$$DeptID \rightarrow DeptName$$

Here, DeptName depends on DeptID rather than directly on StudentID. Therefore, the relation is not in 3NF.

To convert it into 3NF:

$$Student(StudentID, DeptID)$$

$$Department(DeptID, DeptName)$$

Thus, 3NF removes transitive dependencies, reduces redundancy, and improves data consistency.

4. What is Redundancy? Explain with Suitable Example. What are the Anomalies Created by Data Redundancy?

Data redundancy refers to the unnecessary repetition of the same data in a database. Redundant data increases storage requirements and may lead to inconsistencies.

Example

SID	Name	DeptName
101	Rahul	CSE
102	Priya	CSE
103	Amit	CSE

Here, the department name "CSE" is stored repeatedly, creating redundancy.

Anomalies Caused by Redundancy

1. Update Anomaly

If the department name changes, it must be updated in multiple rows.

2. Insertion Anomaly

A new department cannot be inserted unless a student record exists.

3. Deletion Anomaly

Deleting the last student of a department may remove department information completely.

Normalization is used to eliminate redundancy and these anomalies, thereby improving database efficiency and consistency.

5. Explain the Classification of Functional Dependency

A Functional Dependency (FD) describes the relationship between attributes in a relation. Functional dependencies are classified into different types based on how attributes depend on each other.

1. Trivial Functional Dependency

A functional dependency is trivial if the dependent attribute is a subset of the determinant attribute.

Example:

$$(A, B) \rightarrow A$$

Since A is a subset of (A,B), the dependency is trivial.

2. Non-Trivial Functional Dependency

A functional dependency is non-trivial if the dependent attribute is not a subset of the determinant.

Example:

$$StudentID \rightarrow Name$$

3. Completely Non-Trivial Functional Dependency

A dependency is completely non-trivial when the determinant and dependent attributes have no common attribute.

Example:

$$StudentID \rightarrow Department$$

4. Full Functional Dependency

An attribute is fully dependent on the entire primary key and not on any subset of it.

Example:

$$(StudentID, CourseID) \rightarrow Marks$$

Thus, the classification of functional dependencies helps in database normalization and schema design.

6. Consider the Relation Schema R(A,B,C,D,E,F) and the Functional Dependencies $A \rightarrow B$, $C \rightarrow DF$, $AC \rightarrow E$, $D \rightarrow F$. Find the Candidate Keys of R

Given:

$$A \rightarrow B$$

$$C \rightarrow DF$$

$$AC \rightarrow E$$

$$D \rightarrow F$$

To find the candidate key, compute the closure of AC.

$$(AC)^+$$

Using

$$A \rightarrow B$$

we get B.

Using

$$C \rightarrow DF$$

we get D and F.

Using

$$AC \rightarrow E$$

we get E.

Therefore,

$$(AC)^+ = \{A, B, C, D, E, F\}$$

which contains all attributes of the relation.

Now check A and C individually:

$$A^+ = \{A, B\}$$

$$C^+ = \{C, D, F\}$$

Neither determines all attributes.

Hence neither A nor C is a candidate key.

Candidate Key

$$AC$$

Therefore, AC is the candidate key of relation R.

7. Define and Explain Fourth Normal Form (4NF) with Suitable Example

A relation is said to be in Fourth Normal Form (4NF) if it is in Boyce-Codd Normal Form (BCNF) and contains no non-trivial multivalued dependency.

A multivalued dependency occurs when one attribute determines multiple independent values of another attribute.

Example

Consider:

$$Student(StudentName, Hobby, Language)$$

A student can have multiple hobbies and multiple languages.

Example data:

Student	Hobby	Language
Rahul	Cricket	English
Rahul	Cricket	Hindi
Rahul	Music	English
Rahul	Music	Hindi

This relation contains multivalued dependencies.

To achieve 4NF, decompose into:

$$StudentHobby(Student, Hobby)$$

$$StudentLanguage(Student, Language)$$

Thus, 4NF removes multivalued dependencies and reduces redundancy.

8. Explain Join Dependencies with Fifth Normal Form (5NF) with an Example

A Join Dependency (JD) exists when a relation can be reconstructed by joining multiple smaller relations without loss of information.

Fifth Normal Form (5NF), also known as Project Join Normal Form (PJNF), eliminates join dependencies.

A relation is in 5NF if every join dependency is implied by the candidate keys.

Example

Consider the relation:

$$SupplierPartProject(Supplier, Part, Project)$$

This relation can be decomposed into:

$$SupplierPart(Supplier, Part)$$

$$SupplierProject(Supplier, Project)$$

$$PartProject(Part, Project)$$

The original relation can be reconstructed by joining these three relations.

Benefits of 5NF:

- Eliminates redundancy.
- Prevents update anomalies.
- Ensures lossless decomposition.

Thus, 5NF deals with join dependencies and ensures maximum normalization.

9. Consider the Relation R(v,w,x,y,z) with Functional Dependencies {z→y, y→z, x→y, x→v, vw→x}. Find the Closure of {xy}

Given Functional Dependencies:

$$z \rightarrow y$$

$$y \rightarrow z$$

$$x \rightarrow y$$

$$x \rightarrow v$$

$$vw \rightarrow x$$

Find:

$$(xy)^+$$

Start with:

$$(xy)^+ = \{x, y\}$$

Using

$$y \rightarrow z$$

add z.

$$\{x, y, z\}$$

Using

$$x \rightarrow v$$

add v.

$$\{x, y, z, v\}$$

Attribute w cannot be derived.

Therefore,

$$(xy)^+ = \{x, y, z, v\}$$

Hence, the closure of $\{xy\}$ is:

$$\{x, y, z, v\}$$

10. Given Below are Two Sets of Functional Dependencies for a Relation R(A,B,C,D,E,H). Are They Equivalent?

Given:

$$F = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$$

$$G = \{A \rightarrow CD, E \rightarrow AH\}$$

Two sets of functional dependencies are said to be equivalent if they imply the same set of dependencies.

Checking F

From

$$A \rightarrow C$$

and

$$AC \rightarrow D$$

Since A determines C, using transitivity:

$$A \rightarrow D$$

Therefore,

$$A \rightarrow CD$$

Similarly,

$$E \rightarrow AD$$

and

$$A \rightarrow C$$

imply

$$E \rightarrow ACD$$

Also,

$$E \rightarrow H$$

Hence,

$$E \rightarrow AH$$

can be derived.

Checking G

From

$$A \rightarrow CD$$

we get

$$A \rightarrow C$$

and

$$A \rightarrow D$$

From

$$E \rightarrow AH$$

we get

$$E \rightarrow A$$

and

$$E \rightarrow H$$

Since

$$A \rightarrow C$$

and

$$A \rightarrow D$$

therefore

$$E \rightarrow AD$$

Thus, all dependencies in F can be obtained from G and vice versa.

Conclusion

Since each set of functional dependencies can be derived from the other,

$$F \equiv G$$

Therefore, the two sets of functional dependencies are equivalent.

11. “All BCNF Relations are in 3NF but not Vice Versa.” Prove the Statement with Example

Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and contains no transitive dependency.

Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if for every non-trivial functional dependency

$$X \rightarrow Y$$

the determinant X must be a super key.

BCNF is stricter than 3NF.

Example

Consider relation:

$$R(Student, Course, Instructor)$$

Functional Dependencies:

$$(Student, Course) \rightarrow Instructor$$

$$Instructor \rightarrow Course$$

Candidate Key:

$$(Student, Course)$$

The relation satisfies 3NF because the dependent attribute Course is part of a candidate key.

However,

$$Instructor \rightarrow Course$$

violates BCNF because Instructor is not a super key.

Therefore, the relation is in 3NF but not in BCNF.

Conclusion

Every BCNF relation automatically satisfies the conditions of 3NF. However, some relations may satisfy 3NF but violate BCNF. Hence,

$$BCNF \subset 3NF$$

and therefore all BCNF relations are in 3NF but not vice versa.

12. Explain Multivalued Dependency with Suitable Example

A Multivalued Dependency (MVD) occurs when one attribute in a relation determines multiple independent values of another attribute.

It is represented as:

$$AB$$

which means that for a given value of A, there may be multiple values of B independent of other attributes.

Example

Consider the relation:

Student(StudentName, Hobby, Language)

Suppose Rahul has two hobbies and two languages.

Student	Hobby	Language
Rahul	Cricket	English
Rahul	Cricket	Hindi
Rahul	Music	English
Rahul	Music	Hindi

Here,

StudentNameHobby

and

StudentNameLanguage

The hobbies and languages are independent of each other, causing redundancy. To remove this redundancy, the relation can be decomposed into:

StudentHobby(StudentName, Hobby)

StudentLanguage(StudentName, Language)

Advantages of Removing MVD

- Reduces redundancy.
- Eliminates update anomalies.
- Improves database consistency.

Thus, multivalued dependencies are removed by converting the relation into Fourth Normal Form (4NF).

MODULE - IV - 10 Marks

1. Consider a Schema R(A,B,C,D) and Functional Dependencies $A \rightarrow B$ and $C \rightarrow D$. Check Whether the Decomposition of R into R1(AB) and R2(CD) Gives Lossless Join and Dependency Preserving Decomposition

Given relation:

$$R(A, B, C, D)$$

Functional Dependencies:

$$A \rightarrow B$$

$$C \rightarrow D$$

The relation is decomposed into:

$$R_1(A, B)$$

$$R_2(C, D)$$

1. Checking Lossless Join Property

A decomposition of a relation R into R1 and R2 is lossless if:

$$R_1 \cap R_2$$

contains a candidate key of either R1 or R2.

Here,

$$R_1 \cap R_2 = \emptyset$$

since there is no common attribute between R1 and R2.

For a lossless decomposition, the common attributes should functionally determine all attributes of at least one relation.

Since there is no common attribute,

$$R_1 \cap R_2 = \emptyset$$

the lossless join condition is not satisfied.

Therefore, the decomposition is **Lossy**.

Example

Suppose

$$R = \begin{array}{|c|c|c|c|} \hline A & B & C & D \\ \hline 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ \hline \end{array}$$

After decomposition:

$$R_1 = \begin{array}{|c|c|} \hline A & B \\ \hline 1 & 2 \\ 5 & 6 \\ \hline \end{array}$$

$$R_2 = \begin{array}{|c|c|} \hline C & D \\ \hline 3 & 4 \\ 7 & 8 \\ \hline \end{array}$$

Natural join of R1 and R2 produces extra tuples because there is no common attribute.
Hence the decomposition is lossy.

2. Checking Dependency Preservation

A decomposition is dependency preserving if all functional dependencies can be enforced on the decomposed relations without joining them.

For

$$R_1(A, B)$$

the dependency

$$A \rightarrow B$$

is preserved.

For

$$R_2(C, D)$$

the dependency

$$C \rightarrow D$$

is preserved.

Thus all original functional dependencies are preserved in the decomposed relations.

Conclusion

- Lossless Join Property : **Not Satisfied (Lossy Decomposition)**
- Dependency Preservation : **Satisfied**

Therefore, the decomposition

$$R(A, B, C, D) \rightarrow R_1(A, B), R_2(C, D)$$

is dependency preserving but not lossless.

2. Consider a Relation R(A,B,C,D,E,F,G) with the Functional Dependencies

$$AB \rightarrow CD$$

$$AF \rightarrow D$$

$$DE \rightarrow F$$

$$C \rightarrow G$$

$$F \rightarrow E$$

$$G \rightarrow A$$

Find the closure of attribute sets {BG} and {AF}. Illustrate Fully Functional Dependency with example.

(i) Closure of BG

Start with:

$$(BG)^+ = \{B, G\}$$

Using

$$G \rightarrow A$$

add A:

$$\{A, B, G\}$$

Using

$$AB \rightarrow CD$$

add C and D:

$$\{A, B, C, D, G\}$$

Using

$$C \rightarrow G$$

G is already present.

No further attributes can be obtained.

Therefore,

$$(BG)^+ = \{A, B, C, D, G\}$$

Hence BG is not a candidate key since E and F are missing.

(ii) Closure of AF

Start with:

$$(AF)^+ = \{A, F\}$$

Using

$$AF \rightarrow D$$

add D:

$$\{A, F, D\}$$

Using

$$F \rightarrow E$$

add E:

$$\{A, F, D, E\}$$

Using

$$DE \rightarrow F$$

F already exists.

No other dependencies can be applied.

Therefore,

$$(AF)^+ = \{A, D, E, F\}$$

Hence AF is not a candidate key because B, C, and G are missing.

Fully Functional Dependency

An attribute is said to be fully functionally dependent on a key if it depends on the entire key and not on any proper subset of that key.

If

$$X \rightarrow Y$$

and Y cannot be determined by any subset of X, then Y is fully functionally dependent on X.

Example

Consider the relation:

$$ENROLL(StudentID, CourseID, Marks)$$

Functional Dependency:

$$(StudentID, CourseID) \rightarrow Marks$$

Marks depends on both StudentID and CourseID together.

Neither

$$StudentID \rightarrow Marks$$

nor

$$CourseID \rightarrow Marks$$

holds.

Therefore Marks is fully functionally dependent on the composite key (StudentID, CourseID).

Conclusion

$$(BG)^+ = \{A, B, C, D, G\}$$

$$(AF)^+ = \{A, D, E, F\}$$

A fully functional dependency exists when an attribute depends on the entire key and not on a part of it. Such dependencies are required for a relation to satisfy Second Normal Form (2NF).

3. Consider the Relation

Shipping(Shipname, Shiptype, Voyageid, Cargo, Port, Date)

Functional Dependencies:

$$Shipname \rightarrow Shiptype$$

$$Voyageid \rightarrow Shipname, Cargo$$

$$Shipname \rightarrow Shiptype$$

$$(Date, Shipname) \rightarrow Voyageid, Port$$

(a) Identify the Candidate Keys

Compute the closure of {Date, Shipname}.

Initially,

$$(Date, Shipname)^+ = \{Date, Shipname\}$$

Using

$$(Date, Shipname) \rightarrow Voyageid, Port$$

add Voyageid and Port.

$$= \{Date, Shipname, Voyageid, Port\}$$

Using

$$Voyageid \rightarrow Shipname, Cargo$$

add Cargo.

$$= \{Date, Shipname, Voyageid, Port, Cargo\}$$

Using

$$Shipname \rightarrow Shiptype$$

add Shiptype.

$$= \{Date, Shipname, Voyageid, Port, Cargo, Shiptype\}$$

All attributes are obtained.

Therefore,

$$(Date, Shipname)$$

is a candidate key.

(b) Normalize the Relation

First Normal Form (1NF)

The given relation already contains atomic values. Therefore it is in 1NF.

Second Normal Form (2NF)

Candidate Key:

$$(Date, Shipname)$$

Observe:

$$Shipname \rightarrow Shiptype$$

Shiptype depends only on a part of the candidate key (Shipname).

Therefore a partial dependency exists.

Decompose into:

$$SHIP(Shipname, Shiptype)$$

$$SHIPPING_1(Shipname, Voyageid, Cargo, Port, Date)$$

Now the relation satisfies 2NF.

Third Normal Form (3NF)

In SHIPPING_1:

$$Voyageid \rightarrow Cargo$$

Cargo depends on Voyageid, which is not a candidate key.

This creates a transitive dependency.

Decompose further:

$$VOYAGE(Voyageid, Cargo)$$

$$SHIP_VOYAGE(Date, Shipname, Voyageid, Port)$$

Final Normalized Relations

$$SHIP(Shipname, Shiptype)$$

$$VOYAGE(Voyageid, Cargo)$$

$$SHIP_VOYAGE(Date, Shipname, Voyageid, Port)$$

Conclusion

The candidate key is:

$(Date, Shipname)$

After removing partial and transitive dependencies, the relation is decomposed into:

$SHIP(Shipname, Shiptype)$

$VOYAGE(Voyageid, Cargo)$

$SHIP_VOYAGE(Date, Shipname, Voyageid, Port)$

which is in Third Normal Form (3NF).

4. Define BCNF. How Does It Differ from 3NF? Why Is It Considered Stronger Than 3NF?

Boyce-Codd Normal Form (BCNF)

A relation is said to be in BCNF if for every non-trivial functional dependency

$$X \rightarrow Y$$

the determinant X must be a super key.

BCNF is a stricter version of Third Normal Form (3NF) and helps eliminate more redundancy from a relation.

Difference Between 3NF and BCNF

Feature	3NF	BCNF
Condition	Allows some exceptions	Determinant must always be a super key
Redundancy	May exist	Reduced further
Strength	Less strict	More strict

Example

Consider relation:

$R(Student, Course, Instructor)$

with functional dependencies:

$(Student, Course) \rightarrow Instructor$

$Instructor \rightarrow Course$

The relation satisfies 3NF but violates BCNF because Instructor is not a super key.

Why BCNF is Stronger Than 3NF?

- Removes more redundancy.
- Eliminates certain anomalies not handled by 3NF.
- Ensures every determinant is a super key.
- Provides a better database design.

Conclusion

BCNF is a stronger form of normalization than 3NF. Every BCNF relation is in 3NF, but every 3NF relation is not necessarily in BCNF.

5. Discuss Armstrong's Axioms

Armstrong's Axioms are a set of rules used to derive new functional dependencies from existing functional dependencies. They were proposed by William W. Armstrong and are widely used in database normalization, finding attribute closures, and identifying candidate keys.

The three basic Armstrong's Axioms are:

1. Reflexivity Rule

If Y is a subset of X , then X functionally determines Y .

$$Y \subseteq X \Rightarrow X \rightarrow Y$$

Example

$$(A, B) \rightarrow A$$

Since A is a subset of (A, B) , the dependency is valid.

2. Augmentation Rule

If

$$X \rightarrow Y$$

then adding the same attribute set Z to both sides gives

$$XZ \rightarrow YZ$$

Example

If

$$A \rightarrow B$$

then

$$AC \rightarrow BC$$

3. Transitivity Rule

If

$$X \rightarrow Y$$

and

$$Y \rightarrow Z$$

then

$$X \rightarrow Z$$

Example

If

$$StudentID \rightarrow DeptID$$

and

$$DeptID \rightarrow DeptName$$

then

$$StudentID \rightarrow DeptName$$

Derived Rules

Several useful rules can be derived from the basic Armstrong's Axioms.

Union Rule

If $X \rightarrow Y$ and $X \rightarrow Z$, then

$$X \rightarrow YZ$$

Decomposition Rule

If $X \rightarrow YZ$, then

$$X \rightarrow Y$$

and

$$X \rightarrow Z$$

Pseudo-Transitivity Rule

If

$$X \rightarrow Y$$

and

$$YZ \rightarrow W$$

then

$$XZ \rightarrow W$$

Applications of Armstrong's Axioms

- Finding attribute closures.
- Identifying candidate keys.
- Verifying functional dependencies.
- Performing normalization.
- Designing efficient database schemas.

Conclusion

Armstrong's Axioms provide a sound and complete set of inference rules for functional dependencies. They are fundamental tools in database design and normalization and help ensure data consistency and integrity.

6. Consider the Following Relation Schema

Works(Pname, Cname, Salary)

Lives(Pname, Street, City)

Located_in(Cname, City)

Manager(Pname, Mgrname)

Write SQL Queries for the Following.

(i) Find the Names of All Persons Who Live in the City Bangalore

```
SELECT Pname
FROM Lives
WHERE City = 'Bangalore';
```

Explanation

The query retrieves the names of all persons whose city is Bangalore from the Lives relation.

(ii) Retrieve the Names of All Persons of "Infosys" Whose Salary is Between Rs. 50000 and Rs. 100000

```
SELECT Pname
FROM Works
WHERE Cname = 'Infosys'
AND Salary BETWEEN 50000 AND 100000;
```

Explanation

The query selects employees working in Infosys whose salary falls within the specified range.

(iii) Find the Names of All Persons Who Live and Work in the Same City

```
SELECT L.Pname
FROM Lives L, Works W, Located_in LI
WHERE L.Pname = W.Pname
AND W.Cname = LI.Cname
AND L.City = LI.City;
```

Explanation

The query compares the city where a person lives with the city where the company is located.

(iv) List the Names of the People Who Work for "Tech M" Along with the Cities They Live In

```
SELECT W.Pname, L.City
FROM Works W, Lives L
WHERE W.Pname = L.Pname
AND W.Cname = 'Tech M';
```

Explanation

The query joins Works and Lives relations and displays employee names along with their residential cities.

(v) Find the Average Salary of "Infosys" Employees

```
SELECT AVG(Salary)
FROM Works
WHERE Cname = 'Infosys';
```

Explanation

The AVG() aggregate function calculates the average salary of employees working in Infosys.

Conclusion

The above SQL queries demonstrate the use of selection, joins, conditions, and aggregate functions to retrieve information from multiple relations.

MODULE - V - 5 Marks

1. Explain the Pros and Cons of Sequential File Organization

Sequential File Organization stores records one after another in a sequential order, usually based on a key field. Records are accessed in the same order in which they are stored.

Advantages

- Simple and easy to implement.
- Efficient for batch processing applications.
- Suitable for sequential access of records.
- Requires less storage overhead.
- Good performance when most records need to be processed.

Disadvantages

- Slow random access to records.
- Insertion of new records is difficult.
- Deletion may require file reorganization.

- Searching for a specific record can be time-consuming.
- Not suitable for frequently updated databases.

Conclusion

Sequential file organization is suitable for applications where records are processed in sequence, but it is less efficient for random access and frequent updates.

2. Write the Differences Between B-Tree and B+ Tree

B-Tree and B+ Tree are balanced tree indexing structures used in database systems for efficient searching and indexing.

Feature	B-Tree	B+ Tree
Data Storage	Stored in all nodes	Stored only in leaf nodes
Search Path	May end at internal node	Always ends at leaf node
Leaf Nodes	Not linked	Linked together
Range Queries	Less efficient	More efficient
Fan-out	Smaller	Larger
Disk Access	More	Less

Conclusion

B+ Tree is preferred in database systems because it supports faster searching and efficient range queries.

3. Explain the Concept of Hash File Organization with Suitable Diagram

Hash File Organization stores records at locations determined by a hash function. The hash function converts a search key into a storage address.

Working

A hash function $H(K)$ is applied to the key K .

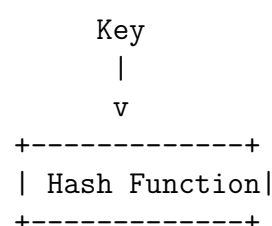
$$Address = H(Key)$$

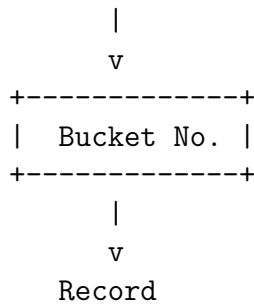
Example:

$$H(105) = 5$$

The record with key 105 is stored in bucket 5.

Diagram





Advantages

- Fast record retrieval.
- Efficient equality search.
- Simple implementation.

Disadvantages

- Collision may occur.
- Not suitable for range queries.
- Performance depends on the hash function.

4. Define Seek Time, Access Time and Rotational Latency

These terms are used to measure the performance of secondary storage devices such as hard disks.

Seek Time

Seek time is the time required for the disk arm to move the read/write head to the desired track.

Rotational Latency

Rotational latency is the time taken for the required sector to rotate under the read/write head.

Access Time

Access time is the total time required to access data from the disk.

$$\text{Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$$

Importance

- Determines disk performance.
- Lower access time means faster data retrieval.
- Important for database operations.

5. Compare Primary Indexing and Clustering Indexing

Primary indexing and clustering indexing are indexing techniques used to improve data retrieval speed.

Feature	Primary Index	Clustering Index
Based On	Primary key	Non-key attribute
Uniqueness	Unique values	May contain duplicates
Number Allowed	Only one	One per clustering field
Data Ordering	Ordered on primary key	Ordered on clustering field
Purpose	Fast record access	Group similar records together

Conclusion

Primary indexing is used on unique key attributes, whereas clustering indexing is used to organize records having similar attribute values.

6. Disadvantages of B-Tree over B+ Tree

Although B-Tree is an efficient indexing structure, it has several disadvantages compared to B+ Tree.

- Data is stored in both internal and leaf nodes.
- Lower fan-out results in a taller tree.
- More disk accesses may be required.
- Sequential traversal is less efficient.
- Range queries are slower.
- Leaf nodes are not linked together.

Conclusion

Because of these limitations, B+ Tree is generally preferred in modern database management systems.

7. Explain How Secondary Indexing Can Help in Join Operation

A secondary index is an index created on a non-ordering attribute of a file. It provides an additional access path to records.

During join operations, secondary indexes can be created on the join attributes of relations.

Benefits in Join Operations

- Speeds up searching of matching records.
- Reduces the number of disk accesses.
- Improves join performance.

- Avoids scanning entire tables.

Example

Consider:

Student(StudentID, Name, DeptID)

Department(DeptID, DeptName)

If a secondary index exists on DeptID, matching records can be located quickly during the join operation.

Conclusion

Secondary indexing significantly improves the efficiency of join operations by reducing search time and disk I/O operations.

MODULE - V - 10 Marks

1. Explain the Concept of Sequential File Organization with Suitable Diagram

Sequential File Organization is a file organization technique in which records are stored one after another in a sequential order based on a key field. Records are generally arranged in ascending or descending order of the primary key.

In this organization, records are accessed sequentially from the beginning of the file until the desired record is found. It is widely used in applications where most records are processed in sequence, such as payroll systems, billing systems, and inventory management.

Working Principle

Suppose student records are stored according to StudentID.

StudentID	Name
101	Rahul
102	Priya
103	Amit
104	Neha
105	Ankit

Records are physically stored in the same order.

Diagram

```

+-----+ +-----+ +-----+ +-----+ +-----+
| 101 | -> | 102 | -> | 103 | -> | 104 | -> | 105 |
+-----+ +-----+ +-----+ +-----+ +-----+

```

Advantages

- Simple and easy to implement.
- Efficient for batch processing.

- Suitable for sequential access of records.
- Requires less storage overhead.
- Good performance when most records are processed together.

Disadvantages

- Slow random access.
- Insertion of new records is difficult.
- Deletion may require reorganization.
- Searching can be time-consuming for large files.
- Not suitable for frequently updated databases.

Applications

- Payroll systems
- Banking applications
- Inventory management
- Student record systems

Conclusion

Sequential File Organization stores records in sorted order and is efficient for sequential processing. However, it is less suitable for applications requiring frequent insertions, deletions, or random access.

2. Construct B+ Tree for the Insertion Sequence 2,3,5,11,17,19,23,29,31 When the Number of Pointers Per Node is 4

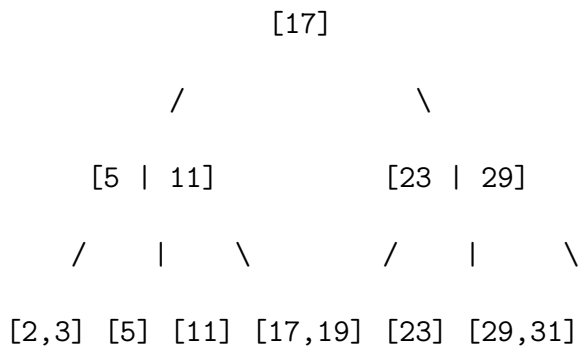
For a B+ Tree of order 4:

- Maximum children = 4
- Maximum keys in an internal node = 3
- Maximum keys in a leaf node = 3

The keys are inserted in the order:

2, 3, 5, 11, 17, 19, 23, 29, 31

After performing the required node splits, the final B+ Tree is:



Leaf Node Links:

[2,3] <-> [5] <-> [11] <-> [17,19]
 <-> [23] <-> [29,31]

Explanation

1. Insert 2, 3, 5 into the first leaf node.
2. Inserting 11 causes the first split.
3. Insert 17 and 19, causing another split.
4. Insert 23 and 29, resulting in further leaf splits.
5. Insert 31 and perform the final adjustment.
6. The middle key 17 becomes the root key.
7. Internal nodes store only search keys, while actual records are stored in leaf nodes.
8. All leaf nodes remain at the same level and are linked sequentially.

Properties Verified

- All leaves are at the same level.
- Internal nodes contain only index keys.
- Leaf nodes contain actual data entries.
- Leaf nodes are connected using pointers for efficient range queries.

Hence, the above structure is the final B+ Tree for the given insertion sequence.

3. Given a Block Can Hold Either 3 Records or 10 Key Pointers. A Database Contains n Records. Determine the Number of Blocks Required for the Data File and the Dense Index.

Given

- One block can store 3 records.
- One index block can store 10 key-pointer entries.
- Total number of records = n .

(i) Number of Blocks Required for the Data File

Since one block can hold 3 records, the number of blocks required to store n records is:

$$DataFileBlocks = \left\lceil \frac{n}{3} \right\rceil$$

where $\lceil x \rceil$ denotes the ceiling function.

(ii) Number of Blocks Required for a Dense Index

In a dense index, there is one index entry for every record. Therefore,

$$NumberOfIndexEntries = n$$

Since one index block can store 10 key-pointer entries, the number of index blocks required is:

$$DenseIndexBlocks = \left\lceil \frac{n}{10} \right\rceil$$

Example

Let

$$n = 300$$

Then,

$$DataFileBlocks = \frac{300}{3} = 100$$

and

$$DenseIndexBlocks = \frac{300}{10} = 30$$

Thus, 100 blocks are required for the data file and 30 blocks are required for the dense index.

Conclusion

For a database containing n records:

$$DataFileBlocks = \left\lceil \frac{n}{3} \right\rceil$$

$$DenseIndexBlocks = \left\lceil \frac{n}{10} \right\rceil$$

These formulas give the number of blocks needed to store the data file and the dense index respectively.

4. The Order of a Leaf Node in a B+ Tree is the Maximum Number of (Value, Data Record Pointer) Pairs it Can Hold. Given that the Block Size is 1K Bytes, Data Record Pointer is 7 Bytes Long, the Value Field is 9 Bytes Long and a Block Pointer is 6 Bytes Long, What is the Order of the Leaf Node?

Given

- Block Size = 1 KB = 1024 bytes
- Data Record Pointer = 7 bytes
- Value Field = 9 bytes
- Block Pointer = 6 bytes

Let the order of the leaf node be p .

A leaf node contains:

- p pairs of (Value, Record Pointer)
- One block pointer to the next leaf node

Size of one (Value, Record Pointer) pair:

$$9 + 7 = 16\text{bytes}$$

Therefore, total space required is:

$$16p + 6 \leq 1024$$

Solving,

$$16p \leq 1018$$

$$p \leq \frac{1018}{16}$$

$$p \leq 63.625$$

Since the order must be an integer,

$$p = 63$$

Verification

$$16 \times 63 + 6 = 1008 + 6 = 1014\text{bytes}$$

which is less than 1024 bytes.

For $p = 64$,

$$16 \times 64 + 6 = 1030 \text{ bytes}$$

which exceeds the block size.

Conclusion

Hence, the maximum number of (Value, Record Pointer) pairs that can be stored in a leaf node is:

$$63$$

Therefore, the order of the leaf node is

$$63$$

MODULE - VI - 5 Marks

1. Explain Serial Schedule with Example

A serial schedule is a schedule in which transactions are executed one after another without any interleaving of operations. One transaction completes entirely before the next transaction begins.

Example

Consider two transactions:

$$T_1 : R(A), W(A)$$

$$T_2 : R(B), W(B)$$

A serial schedule is:

$$R_1(A) \ W_1(A) \ R_2(B) \ W_2(B)$$

Here, transaction T_1 completes before T_2 starts.

Advantages

- Easy to understand and implement.
- Ensures database consistency.
- No concurrency-related problems.

Disadvantage

Database resources are not utilized efficiently because transactions execute one at a time.

Thus, a serial schedule guarantees correctness but provides less concurrency.

2. Explain Non-Serial Schedule with Example

A non-serial schedule is a schedule in which operations of multiple transactions are interleaved. Transactions execute concurrently rather than one after another.

Example

Consider two transactions:

$$T_1 : R(A), W(A)$$

$$T_2 : R(B), W(B)$$

A non-serial schedule is:

$$R_1(A) \ R_2(B) \ W_1(A) \ W_2(B)$$

Here, operations of T_1 and T_2 are interleaved.

Advantages

- Better CPU and resource utilization.
- Higher throughput.
- Supports concurrent transaction execution.

Disadvantages

- May lead to inconsistency.
- Can cause conflicts among transactions.
- Requires concurrency control mechanisms.

Thus, non-serial schedules improve performance but must be carefully controlled.

3. Explain Transaction Control Language (TCL) with Suitable Commands

Transaction Control Language (TCL) consists of SQL commands used to manage database transactions. TCL ensures that database changes are permanently saved or undone when necessary.

The main TCL commands are:

1. COMMIT

The COMMIT command permanently saves all changes made during the transaction.

Example:

```
UPDATE Account
SET Balance = Balance - 1000
WHERE AccNo = 101;
```

```
COMMIT;
```

2. ROLLBACK

The ROLLBACK command cancels all changes made during the current transaction.
Example:

```
ROLLBACK;
```

3. SAVEPOINT

A SAVEPOINT creates a temporary checkpoint within a transaction.
Example:

```
SAVEPOINT SP1;
```

4. RELEASE SAVEPOINT

Removes a previously created savepoint.

Conclusion

TCL commands help maintain database consistency by controlling transaction execution and recovery.

4. Define Lock. Discuss Two-Phase Locking Protocol with Suitable Diagram

Lock

A lock is a mechanism used in DBMS to control concurrent access to data items. It prevents multiple transactions from accessing the same data simultaneously in a way that may lead to inconsistency.

Before accessing a data item, a transaction must obtain a lock on it.

The two common types of locks are:

- **Shared Lock (S-lock)** – Used for reading data.
- **Exclusive Lock (X-lock)** – Used for writing or updating data.

Two-Phase Locking (2PL) Protocol

The Two-Phase Locking Protocol is a concurrency control technique that ensures conflict serializability.

According to this protocol, every transaction has two phases:

1. Growing Phase
2. Shrinking Phase

1. Growing Phase

- The transaction may acquire locks.
- No lock can be released.

2. Shrinking Phase

- The transaction releases locks.

- No new lock can be acquired.

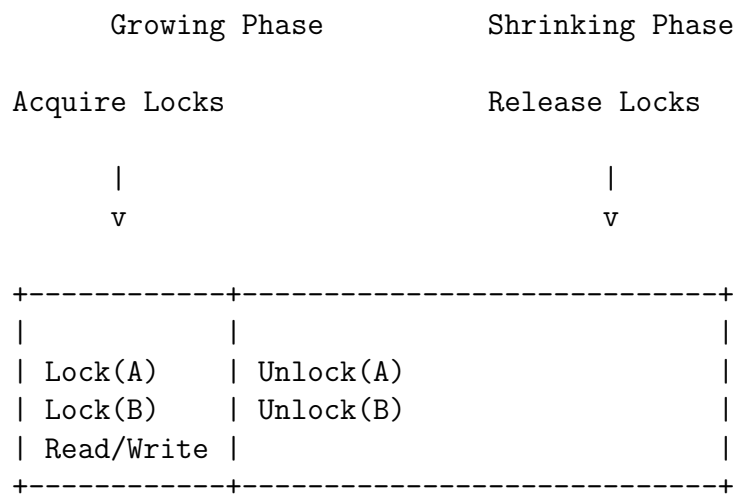
Example

Consider Transaction T_1 :

$Lock(A) \rightarrow Lock(B) \rightarrow Read(A) \rightarrow Write(B) \rightarrow Unlock(A) \rightarrow Unlock(B)$

Locks are acquired first and released later, satisfying the 2PL protocol.

Diagram



Advantages of 2PL

- Ensures conflict serializability.
- Maintains database consistency.
- Prevents lost updates.
- Supports concurrent execution of transactions.

Disadvantages of 2PL

- May cause deadlocks.
- Transactions may wait for locks.
- Increased overhead due to lock management.

Conclusion

Two-Phase Locking is one of the most widely used concurrency control protocols. It divides transaction execution into growing and shrinking phases and ensures serializable schedules while maintaining database consistency.

5. When a Schedule is Called Non-Recoverable Schedule? Explain.

A schedule is said to be a **Non-Recoverable Schedule** if a transaction commits before the transaction from which it has read data commits.

In such a schedule, if the first transaction fails after the second transaction has committed, the database cannot be recovered properly.

Example

Consider two transactions:

$$T_1 : W_1(A)$$

$$T_2 : R_2(A)$$

Schedule:

$$W_1(A) \ R_2(A) \ C_2 \ Abort_1$$

Here,

- T_1 writes data item A.
- T_2 reads A before T_1 commits.
- T_2 commits.
- Later T_1 aborts.

Now T_2 has already committed using incorrect data written by T_1 . Therefore the database cannot be recovered.

Problems of Non-Recoverable Schedule

- Causes inconsistency in the database.
- Recovery becomes impossible.
- Violates transaction reliability.

Conclusion

A non-recoverable schedule should be avoided because committed transactions may depend on uncommitted data, leading to incorrect database states.

6. Differentiate Between Serial Schedule and Serializable Schedule. Explain Different Types of Locks.

Difference Between Serial Schedule and Serializable Schedule

Feature	Serial Schedule	Serializable Schedule
Execution	Transactions execute one after another	Transactions may execute concurrently
Interleaving	Not allowed	Allowed
Concurrency	Low	High
Performance	Lower	Better
Correctness	Always correct	Equivalent to a serial schedule

Example of Serial Schedule

$$R_1(A) \ W_1(A) \ R_2(B) \ W_2(B)$$

Example of Serializable Schedule

$$R_1(A) \ R_2(B) \ W_1(A) \ W_2(B)$$

Although operations are interleaved, the result is equivalent to some serial execution.

Types of Locks

Locks are used to control concurrent access to database items.

1. Shared Lock (S-Lock)

- Allows only reading of a data item.
- Multiple transactions can hold shared locks simultaneously.

Example:

$$Read(A)$$

2. Exclusive Lock (X-Lock)

- Allows both reading and writing.
- No other transaction can access the data item while the lock is held.

Example:

$$Write(A)$$

3. Binary Lock

- Has only two states: Locked and Unlocked.
- Simplest form of locking mechanism.

4. Intent Lock

- Used in multiple-granularity locking.
- Indicates a transaction's intention to acquire locks on lower-level objects.

Conclusion

Serial schedules execute transactions sequentially, whereas serializable schedules allow concurrent execution while preserving correctness. Locks help maintain consistency and prevent conflicts among concurrent transactions.

MODULE - VI - 10 Marks

1. Explain the ACID Properties of Transaction with Example

A transaction is a sequence of database operations that performs a single logical unit of work. To ensure reliability and consistency, every transaction must satisfy the ACID properties.

ACID stands for:

- Atomicity
- Consistency
- Isolation
- Durability

1. Atomicity

Atomicity means that a transaction is treated as a single unit. Either all operations are completed successfully or none of them are performed.

Example:

In a bank transfer, money must be deducted from one account and added to another. If any step fails, the entire transaction is rolled back.

2. Consistency

Consistency ensures that a transaction takes the database from one valid state to another valid state.

Example:

If the total balance of two accounts is Rs. 8000 before a transaction, it should remain Rs. 8000 after the transaction.

3. Isolation

Isolation ensures that concurrent transactions do not interfere with each other.

Example:

If two users update the same account simultaneously, each transaction should execute independently without affecting the other.

4. Durability

Durability guarantees that once a transaction is committed, its changes become permanent even if a system failure occurs.

Example:

After a successful bank transfer and COMMIT operation, the updated balances remain stored even after a power failure.

Example of ACID Transaction

```
BEGIN TRANSACTION;
```

```
UPDATE Account  
SET Balance = Balance - 1000  
WHERE AccNo = 101;
```

```
UPDATE Account  
SET Balance = Balance + 1000  
WHERE AccNo = 102;
```

```
COMMIT;
```

Advantages of ACID Properties

- Maintains database consistency.
- Prevents data corruption.
- Supports reliable transaction processing.
- Ensures proper recovery after failures.

Conclusion

The ACID properties are the foundation of transaction management in DBMS. They ensure that transactions are executed correctly, safely, and reliably while maintaining the integrity of the database.

2. Discuss the States of Transaction with Suitable Diagram

A transaction is a sequence of operations performed on a database. During execution, a transaction passes through different states. These states help the DBMS manage transactions and recover from failures.

The main transaction states are:

1. Active
2. Partially Committed
3. Committed
4. Failed
5. Aborted
6. Terminated

1. Active State

The transaction is currently executing its read and write operations.

2. Partially Committed State

The transaction has executed its final statement but the changes are not yet permanently stored.

3. Committed State

The transaction completes successfully and all changes are permanently saved in the database.

4. Failed State

If an error, system crash, or deadlock occurs, the transaction enters the failed state.

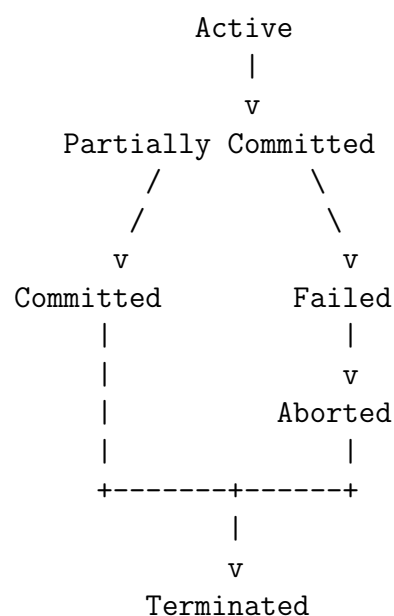
5. Aborted State

The transaction is rolled back and all changes made by it are undone.

6. Terminated State

After commit or abort, the transaction leaves the system and enters the terminated state.

Transaction State Diagram



Importance

- Helps in transaction management.
- Supports recovery from failures.
- Maintains database consistency.
- Ensures reliable execution of transactions.

Conclusion

A transaction moves through Active, Partially Committed, Committed, Failed, Aborted, and Terminated states during its execution. These states help ensure reliable and consistent database operations.

3. Explain Lock-Based Protocols in Details

Lock-based protocols are concurrency control techniques used in DBMS to ensure that multiple transactions can execute safely without causing inconsistency in the database.

A transaction must obtain a lock on a data item before accessing it.

Types of Locks

- **Shared Lock (S-Lock):** Used for reading data. Multiple transactions can hold shared locks simultaneously.
- **Exclusive Lock (X-Lock):** Used for writing data. Only one transaction can hold an exclusive lock at a time.

Two-Phase Locking (2PL) Protocol

The most common lock-based protocol is the Two-Phase Locking protocol.

It consists of two phases:

1. **Growing Phase** – Locks can be acquired but not released.
2. **Shrinking Phase** – Locks can be released but no new locks can be acquired.

Example

$$Lock(A) \rightarrow Read(A) \rightarrow Write(A) \rightarrow Unlock(A)$$

This protocol ensures conflict serializability of schedules.

Advantages

- Maintains database consistency.
- Prevents lost updates.
- Ensures serializable schedules.

Disadvantages

- May cause deadlocks.
- Transactions may have to wait for locks.
- Additional overhead for lock management.

Conclusion

Lock-based protocols are widely used in DBMS to control concurrent access to data and maintain the correctness of transaction execution.

4. Draw the Precedence Graph for the Given Schedule and Test Whether it is Conflict Serializable

Given Schedule:

$$S = \{r_1(x), r_2(y), r_3(y), w_2(y), w_1(x), w_3(x), r_2(x), w_2(x)\}$$

Step 1: Identify Conflicting Operations

Transactions involved:

$$T_1, T_2, T_3$$

Conflicts on x

$$w_1(x) \rightarrow w_3(x)$$

Therefore,

$$T_1 \rightarrow T_3$$

$$w_1(x) \rightarrow r_2(x)$$

Therefore,

$$T_1 \rightarrow T_2$$

$$w_1(x) \rightarrow w_2(x)$$

Therefore,

$$T_1 \rightarrow T_2$$

$$w_3(x) \rightarrow r_2(x)$$

Therefore,

$$T_3 \rightarrow T_2$$

$$w_3(x) \rightarrow w_2(x)$$

Therefore,

$$T_3 \rightarrow T_2$$

Conflicts on y

$$r_3(y) \rightarrow w_2(y)$$

Therefore,

$$T_3 \rightarrow T_2$$

Step 2: Construct the Precedence Graph

Vertices:

$$\{T_1, T_2, T_3\}$$

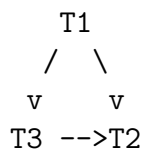
Edges:

$$T_1 \rightarrow T_3$$

$$T_1 \rightarrow T_2$$

$$T_3 \rightarrow T_2$$

Precedence Graph:



Step 3: Check for Cycles

The graph contains the edges:

$$T_1 \rightarrow T_3$$

$$T_1 \rightarrow T_2$$

$$T_3 \rightarrow T_2$$

There is no path from T_2 back to T_1 or T_3 .

Hence, the graph contains **no cycle**.

Step 4: Conflict Serializability

A schedule is conflict serializable if its precedence graph is acyclic.

Since the graph is acyclic, the given schedule is:

ConflictSerializable

Step 5: Serial Order

The topological ordering of the graph is:

$$T_1 \rightarrow T_3 \rightarrow T_2$$

Therefore, the schedule is equivalent to the serial schedule:

$$T_1, T_3, T_2$$

Conclusion

The precedence graph is acyclic. Hence the given schedule is conflict serializable and the correct serial order is:

$$T_1 \rightarrow T_3 \rightarrow T_2$$

MODULE - II - 5 Marks

2. Explain Generalization and Specialization with Example

Generalization and Specialization are important concepts of the Enhanced Entity-Relationship (EER) model. They are used to represent inheritance relationships among entities and help in organizing data efficiently.

Generalization

Generalization is a bottom-up approach in which two or more lower-level entity sets having common attributes are combined into a higher-level entity set called a superclass.

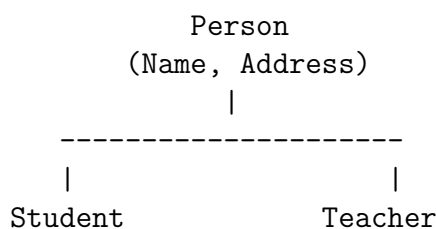
The common properties of the lower-level entities are moved to the higher-level entity, thereby reducing redundancy.

Example

Consider the entities:

- Student(StudentID, Name, Address)
- Teacher(TeacherID, Name, Address)

Both Student and Teacher have common attributes such as Name and Address. Therefore, they can be generalized into a common superclass called Person.



Here, Person is the superclass, while Student and Teacher are subclasses.

Advantages of Generalization

- Reduces redundancy.
- Improves data organization.
- Simplifies database design.
- Promotes attribute sharing.

Specialization

Specialization is a top-down approach in which a higher-level entity set is divided into lower-level entity sets based on specific characteristics.

The lower-level entities inherit all attributes of the higher-level entity and may possess additional attributes.

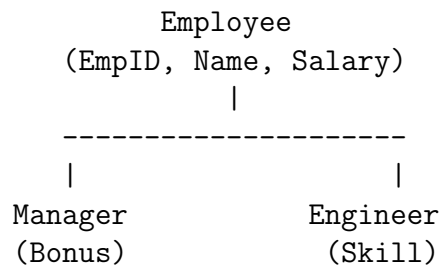
Example

Consider the entity:

- Employee(EmpID, Name, Salary)

Employees can be classified into:

- Manager(EmpID, Name, Salary, Bonus)
- Engineer(EmpID, Name, Salary, Skill)



Manager and Engineer inherit the attributes of Employee and have their own specialized attributes.

Advantages of Specialization

- Represents real-world situations accurately.
- Allows subclass-specific attributes.
- Improves data clarity and flexibility.
- Supports inheritance.

Conclusion

Generalization combines similar entities into a common superclass, whereas specialization divides a superclass into multiple subclasses based on distinguishing characteristics. Both concepts help in reducing redundancy, improving database design, and representing inheritance relationships effectively in the EER model.